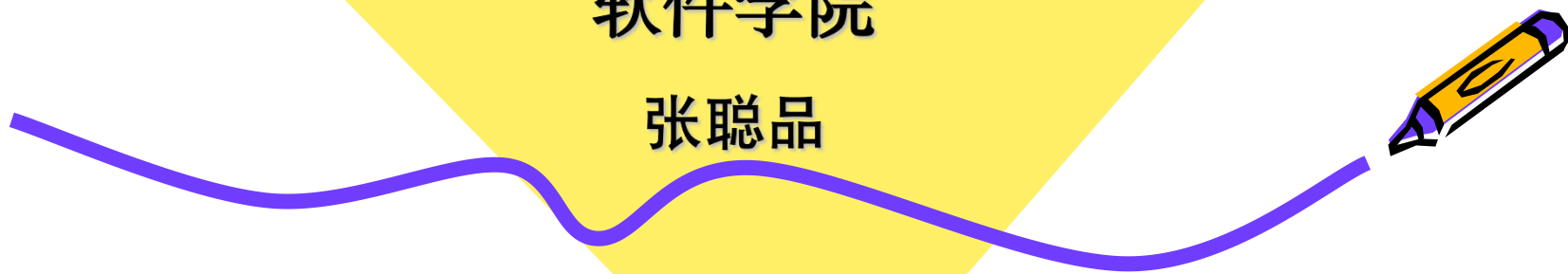




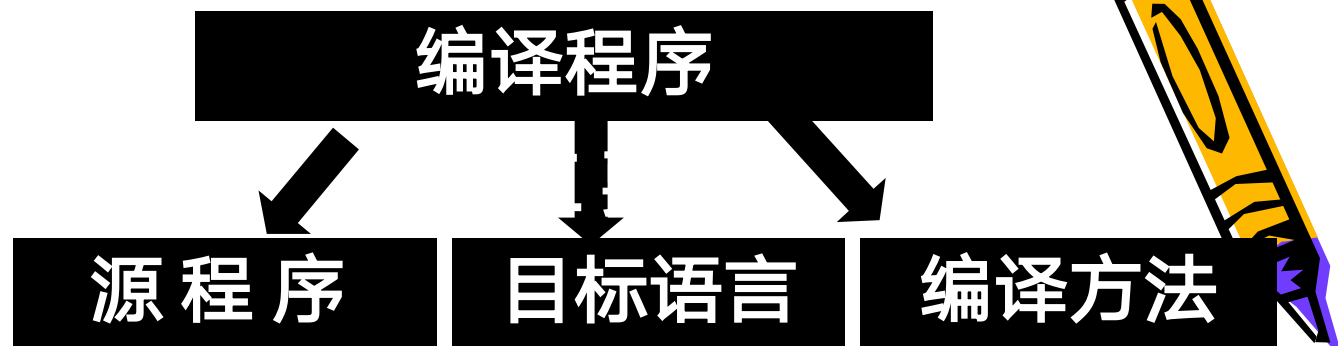
编译原理及实践教程

河南师范大学
软件学院

张聪品



回顾



1 手工构造
词法分析器

2 自动构造词
法分析器

词法分析

语法分析

语义分析与中

间代码生成

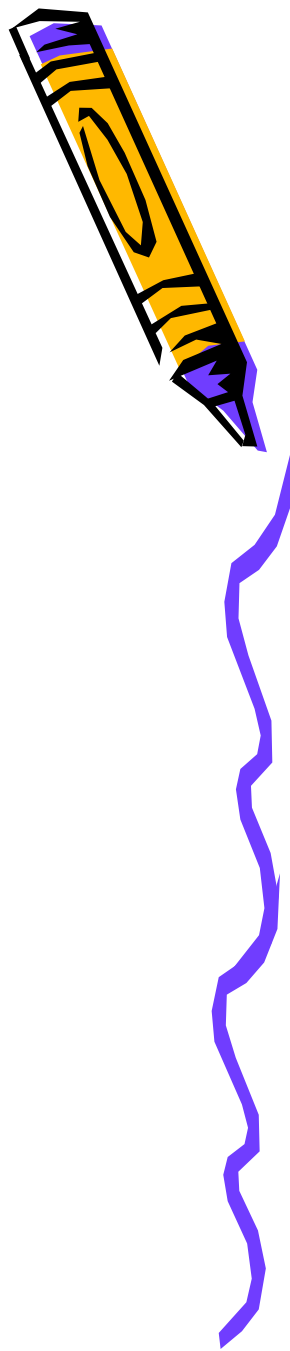
代码优化

目标代码生成

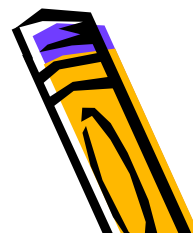


词法分析器的自动生成

- 正规式
- 有穷自动机

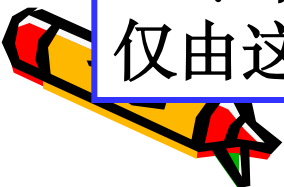


回顾：正规式



■正规式和正规集的递归定义：（设字母表为 Σ ）

- 1、 ϵ 和 ϕ 都是 Σ 上的正规式，表示的正规集为 $\{\epsilon\}$ 和 $\{\}$ ；
- 2、任何 $a \in \Sigma$ ，则 a 是正规式，表示的正规集为 $\{a\}$ ；
- 3、假定 r 和 s 都是 Σ 上的正规式，表示的正规集为 $L(r)$ 和 $L(s)$ ；
 - ◆ (r) 是正规式，表示的正规集为 $L(r)$
 - ◆ $r|s$ 是正规式，表示 $L(r) \cup L(s)$ ；
 - ◆ $r \cdot s$ 是正规式，表示 $L(r)L(s)$ ；
 - ◆ r^* 是正规式，表示 $(L(r))^*$ ；
- ◆ 4、有限次使用上述三步骤而定义的表达式才是 Σ 上的正规式，仅由这些正规式所表示的集合才是 Σ 上的正规集。



回顾：不确定的有穷自动机NFA

- 定义：不确定的有穷自动机NFA也是一个五元组， $M = \{S, \Sigma, \delta, s_0, F\}$ ，其中：
 - S 为有穷状态集，
 - Σ 为有穷输入字母表，
 - δ 为 $S \times \Sigma^*$ 到 S 的幂集 (2^S) 的一种映射：
$$S \times \Sigma^* \rightarrow 2^S$$
 - $s_0 \subseteq S$ 是初始状态集，
 - $F \subseteq S$ 为终止状态集 (可空)。



回顾：NFA的三种表示法（状态图表示）

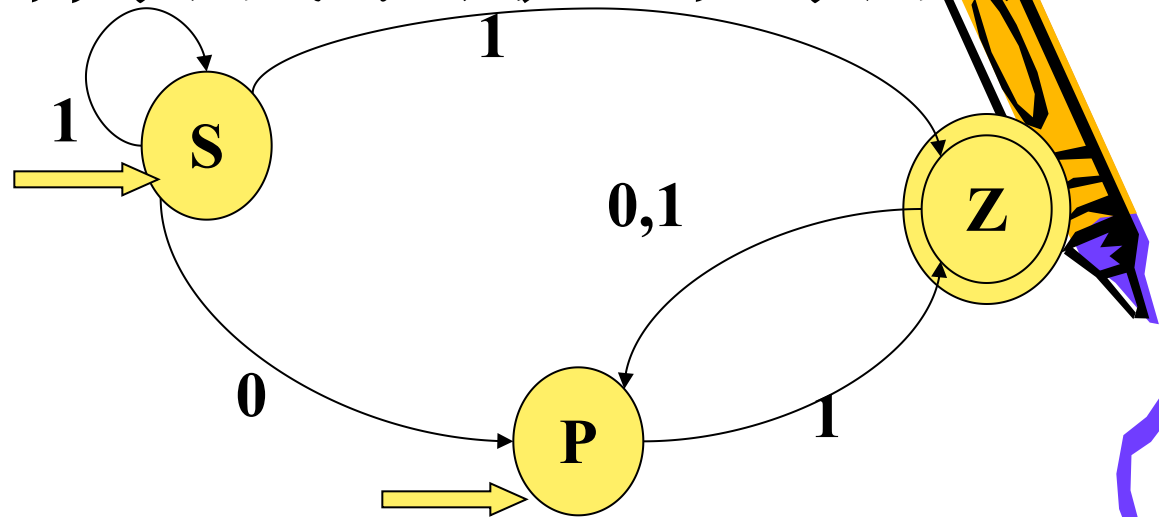
$\delta(S, 0) = \{P\}$

$\delta(S, 1) = \{S, Z\}$

$\delta(Z, 0) = \{P\}$

$\delta(Z, 1) = \{P\}$

$\delta(P, 1) = \{Z\}$



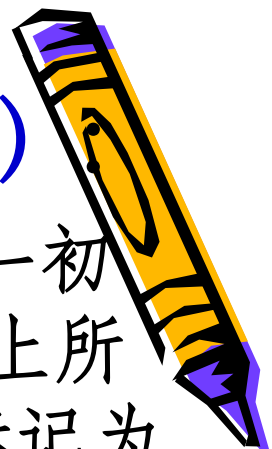
一个含有 m 个状态， n 个输入字符的NFA的状态转换图：
有 m 个结点，每个结点可射出若干条弧与别的结点相连接，
每条弧用一个输入符号或 ε 来标记表示（这些字可以相同，
也可以是 ε ）。

整个图至少有一个初始结点以及若干个（可以是0个）终态
结点，某些结点既可以是初态结点，又可以是终态结点。



Σ^* 上的符号串 t 被NFA M 接受（识别）

- 对于 Σ^* 中的任何一个串 t ，若存在一条从某一初态结点到某一终态结点的通路，且这条通路上所有弧的标记字依序连接成的串（不理睬那些标记为 ε 的弧）等于 t ，则称 t 可为NFA M 所识别（读出或接受）。
- 若 M 的某些结点既是初态结点又是终态结点；或者存在一条从某个初态结点到某个终态结点的道路，其上所有弧的标记均为 ε ，那么空字 ε 可为 M 所接受。
- NFA M 所能识别的符号串的全体记为 $L(M)$ 。



回顾：NFA的不确定性



NFA的特点是它的不确定性，即在当前状态下，读入同一个符号，可能有多个下一状态：

- 在NFA的定义中，就是 δ 函数是一对多的；
- 反映在状态转换图中，就是从一个结点出发，可能有多于一条标记相同的弧转移到不同的下一状态；
- 反映在转换表中，就是 $M[i, a]$ 的值不是一个单一状态，而是一个状态集合。



2 确定的有穷自动机DFA

一个确定的有穷自动机 (DFA) M 是一个五元组: $M = (S, \Sigma, \delta, s_0, F)$, 其中:

1. S 是一个有穷集, 它的每个元素称为一个状态;
2. Σ 是一个有穷字母表, 它的每个元素称为一个输入符号, 所以也称 Σ 为输入符号表;
3. δ 是转换函数, 是在 $S \times \Sigma \rightarrow S$ 上的单值映射, $\delta(s, a) = s'$ ($s \in S, s' \in S$), 就意味着, 当前状态为 s , 输入符为 a 时, 将转换为下一个状态 s' , 我们把 s' 称作 s 的一个后继状态;
4. $s_0 \in S$ 是唯一的一个初态;
5. $F \subseteq S$ 是一个终态集(可空), 也称可接受状态或结束状态。

DFA的矩阵表示

例3：有DFA $M = (\{0, 1, 2, 3\}, \{a, b\}, \delta, 0, \{3\})$

δ 为： $\delta(0,a) = 1$ $\delta(0,b) = 2$

$\delta(1,a) = 3$ $\delta(1,b) = 2$

$\delta(2,a) = 1$ $\delta(2,b) = 3$

$\delta(3,a) = 3$ $\delta(3,b) = 3$

行表示状态，列表示输入字符，矩阵元素表示 $\delta(s, a)$ 的值，称为**状态转换矩阵**。

状态 \ 输入	a	b
0	1	2
1	3	2
2	1	3
3	3	3



• DFA的确定性表现在：

- 对任何状态 $s \in S$ ，在读入了输入符号 $a \in \Sigma$ 之后，能够**唯一地确定**下一个状态
- 映射函数 $\delta : S \times \Sigma \rightarrow S$ 是一个**单值函数**
- 从状态转换图来看，若字母表 Σ 含有 n 个输入字符，那么任何一个状态结点**最多有 n 条弧射出**，而且每条弧以一个**不同的输入字符**标记。



- 字符串 α 可为DFA M 所接受（识别）：

对于 Σ^* 中的任何字符串 α ，若存在一条从初态结点到某个终态结点的通路，且这条通路上所有弧的标记符号连接成的字符串等于 α 。

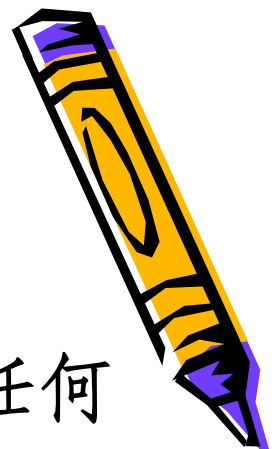
- 若 M 的初态结点又是终态结点，则空字 ε 可为 M 所识别。
- DFA M 所能识别的符号串的全体记为 $L(M)$ 。
- 对于任何两个有穷自动机 M 和 M' ，如果 $L(M)=L(M')$ ，则称 M 与 M' 是等价的。



DFA与NFA的主要区别

- (1) DFA任何状态都没有 ε 转换，即没有任何状态可以不进行输入符号的匹配就直接进入下一个状态；
- (2) DFA对任何状态 s 和任何输入符号 a ，最多只有一条标记为 a 的边离开 s ，即转换函数 $\delta : S \times \Sigma \rightarrow S$ 是一个单值部分函数。
- (3) DFA的初态唯一，NFA的初态为一集合。

DFA是NFA吗？



3 NFA的确定化

- DFA是NFA的特例. 对每个NFA N 一定存在一个DFA M , 使得 $L(M)=L(N)$ 。也就是说: 对每个NFA N 存在着与之等价的DFA M 。
- 方法: (子集法) 将NFA转换成接受同样语言的DFA。
- NFA确定化的基本思路是: DFA的每一个状态对应NFA的一组状态。
- DFA使用它的状态去记录在NFA读入一个输入符号后可能达到的所有状态。



定义



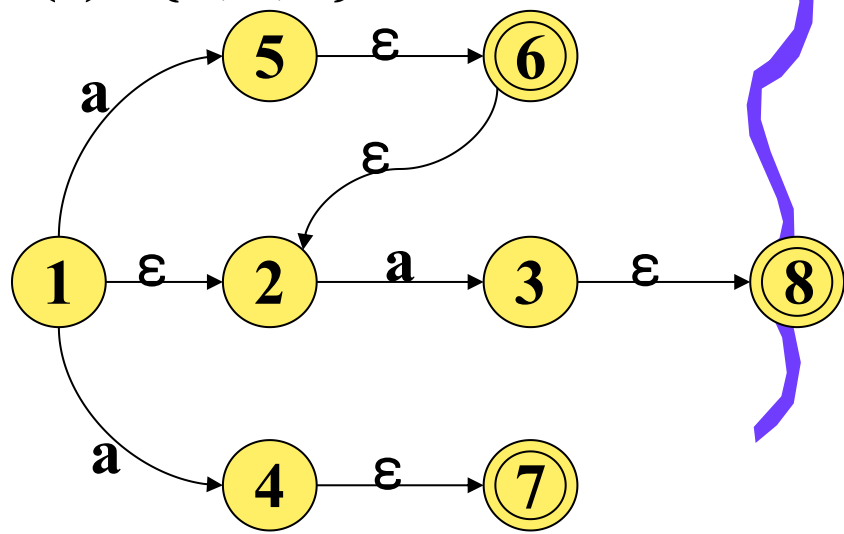
- ❖ 状态集合 I 的 ε -闭包 $\varepsilon\text{-closure}(I)$ ，是一状态集
- ❖ 任何状态 $q \in I$ ，则 $q \in \varepsilon\text{-closure}(I)$ ；
- ❖ 任何状态 $q \in I$ ，则 q 经任意条 ε 弧而能到达的状态 $q' \in \varepsilon\text{-closure}(I)$ 。

例： $I=\{1\}$, $I=\{5\}$,

$\varepsilon\text{-closure}(I)=\{1,2\}$; $\varepsilon\text{-closure}(I)=\{5,6,2\}$;

$I=\{5,4\}$, $\varepsilon\text{-closure}(I)=?$

$\varepsilon\text{-closure}(I)=\{5,4,6,2,7\}$

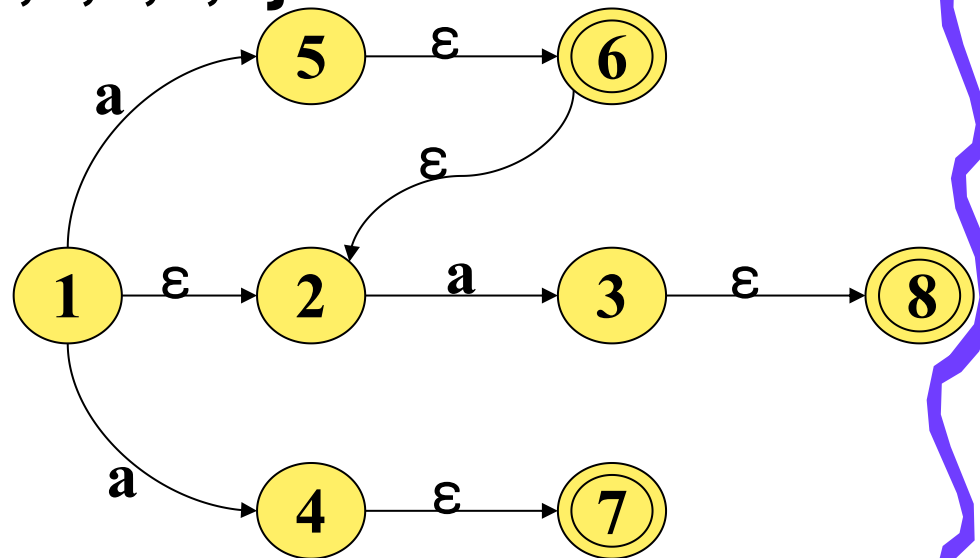


❖ 状态集合I的a弧转换, $I_a = \varepsilon\text{-closure}(J)$,
其中 $J = \text{move}(I, a)$, 即所有可从I中的某一状态
经过一条a弧而到达的状态的全体。

$I = \{1\}$, $J = \text{move}(I, a) = \{5, 4\}$;

$I_a = \varepsilon\text{-closure}(\{5, 4\}) = \{2, 4, 5, 6, 7\}$

$I = \{1, 2\}$, $I_a = ?$



$I = \{1, 2\}$, $J = \text{move}(I, a) = \{5, 3, 4\}$;

$I_a = \varepsilon\text{-closure}(\{5, 3, 4\}) = \{2, 3, 4, 5, 6, 7, 8\}$

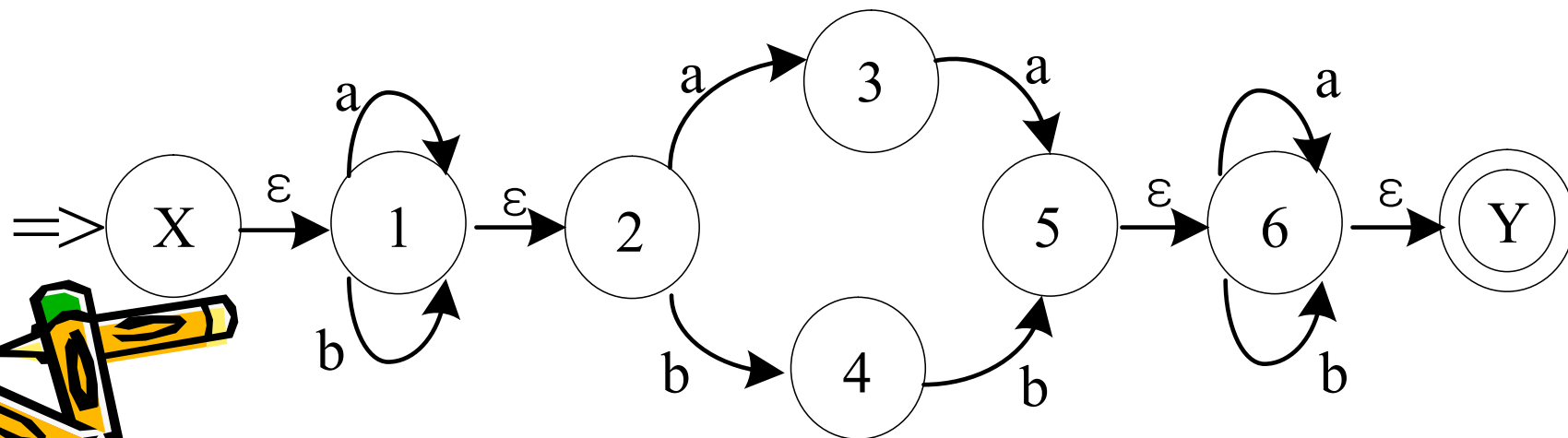
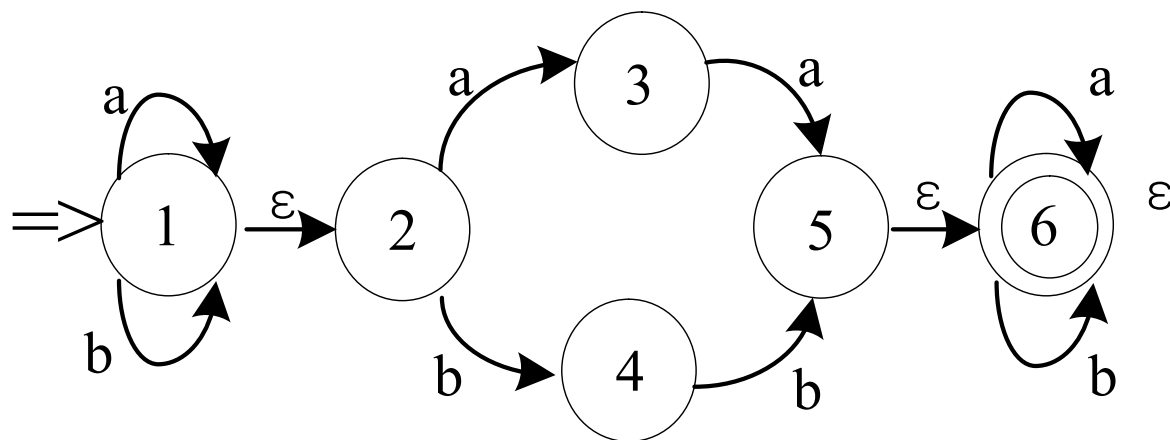
NFA的确定化的步骤



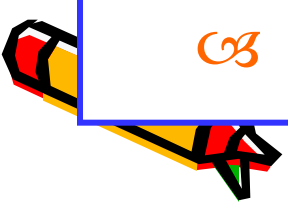
- 第一步：对状态图进行改造
 - (1) 增加状态X, Y, 使之成为新的唯一的初态和终态。从X引 ϵ 弧到原初态结点, 从原终态结点引 ϵ 弧到Y结点。



- 例：有NFA如下：

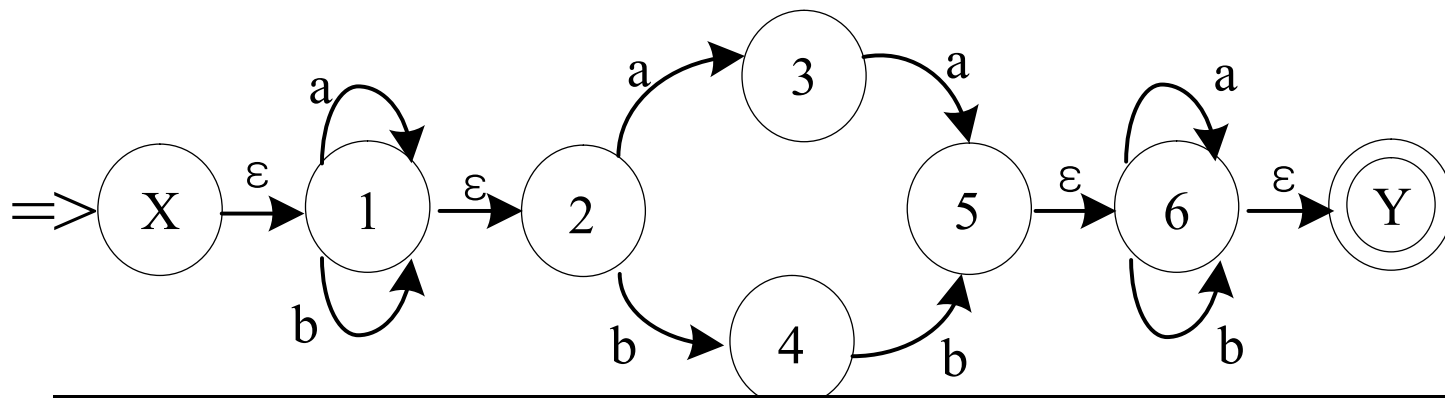


• 第二步：对NFA 进行确定化，构造状态转换表：

1. $\Sigma = \{a_1 \cdots a_k\}$ ，则初始时该表有 $k+1$ 列，分别为 I 、 $Ia_1 \cdots Ia_k$ ，首行首列为 $\varepsilon\text{-closure}(X)$ ，(X 为开始结点)；
 2. 每行的值 $Ia_k = \varepsilon\text{-closure}(\text{move}(I, a))$ ，其行数未知；
- ✎ 将新产生的 Ia_k 集合加入到 I 中作为新的一行，并求该集合 I 的 $Ia_1 \cdots Ia_k$ ，重复之，直到状态不再增加；**初态**就是首行首列的状态，**终态**是含有原终态的集合。
- ✎ 将每个状态子集视为一个新的状态
- 

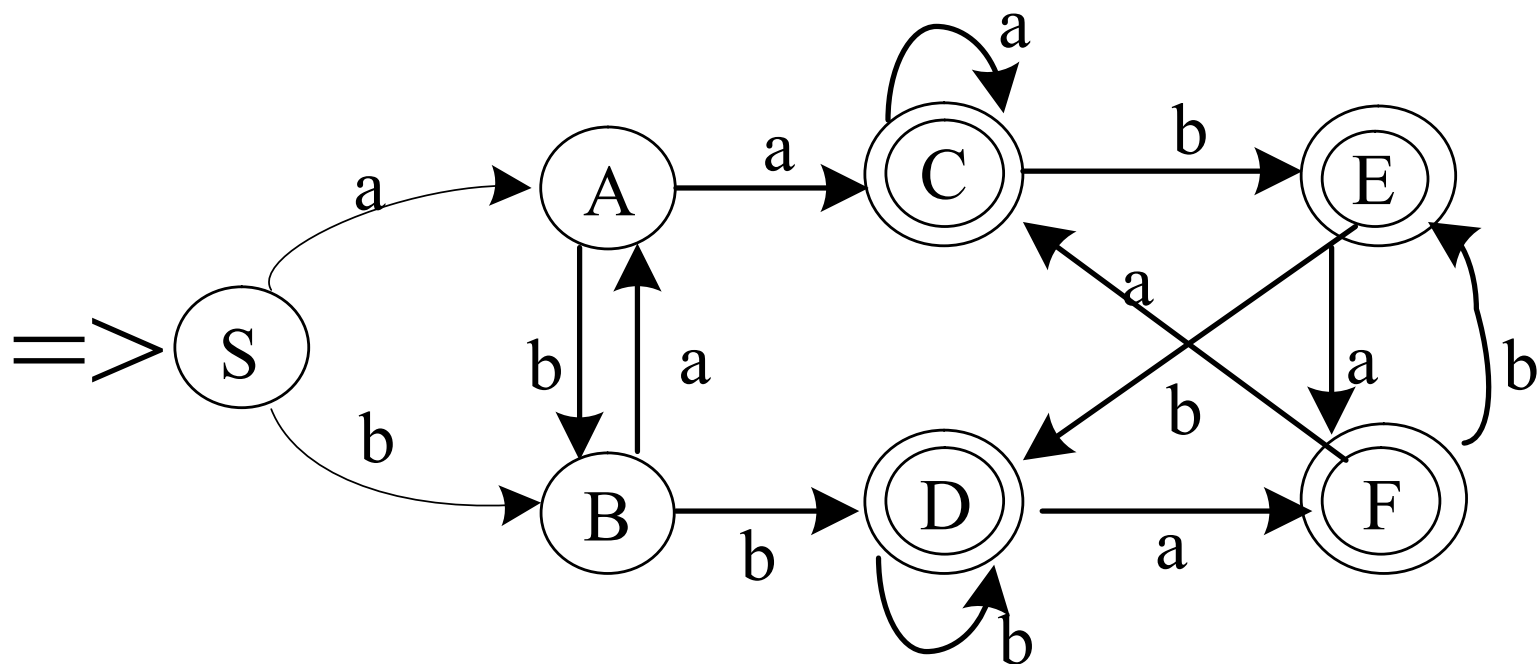
例：将下述NFA确定化

只向后传递一次



I		I_a		I_b	
{x,1,2}	S	{1,2,3}	A	{1,2,4}	B
{1,2,3}	A	{1,2,3,5,6,Y}	C	{1,2,4}	B
{1,2,4}	B	{1,2,3}	A	{1,2,4,5,6,Y}	D
{1,2,3,5,6,Y}	C	{1,2,3,5,6,Y}	C	{1,2,4,6,Y}	E
{1,2,4,5,6,Y}	D	{1,2,3,6,Y}	F	{1,2,4,5,6,Y}	D
{1,2,4,6,Y}	E	{1,2,3,6,Y}	F	{1,2,4,5,6,Y}	D
{1,2,3,6,Y}	F	{1,2,3,5,6,Y}	C	{1,2,4,6,Y}	E

等价的DFA





4 DFA的化简

与某一NFA等价的DFA不一定唯一
不同的DFA识别的正规集可能是相同的

每一个正规集都可以由一个状态数最少的
DFA所识别，这个DFA是唯一的（因状态名不
同的同构情况除外）。



DFA的最小化



- DFA的最小化就是寻求状态数最少的DFA，即：
 - 它没有多余状态；（消去）
 - 它的状态中没有两个是互相等价的。（合并）
- 多余状态是指：从开始状态出发，任何输入串也不能到达的那个状态；或者从这个状态没有通路到达终态。
- 状态S和T等价的条件
 - ◇ 一致性条件 —— 状态S和T必须同时为可接受状态或不可接受状态。
 - ◇ 蔓延性条件 —— 对于所有输入符号，状态S和状态T必须转换到等价的状态里。



DFA的最小化的方法——分割法



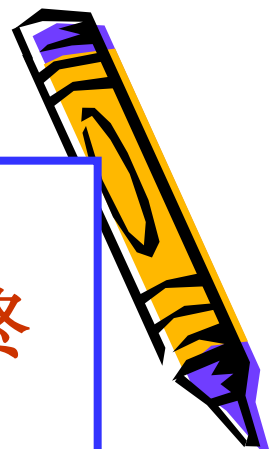
- 分割法的核心

- 把DFA的全部状态划分成一些互不相交的子集，使得任何不同的两子集的状态都是可区别的（不等价），而同一子集中的任何两个状态都是等价的。

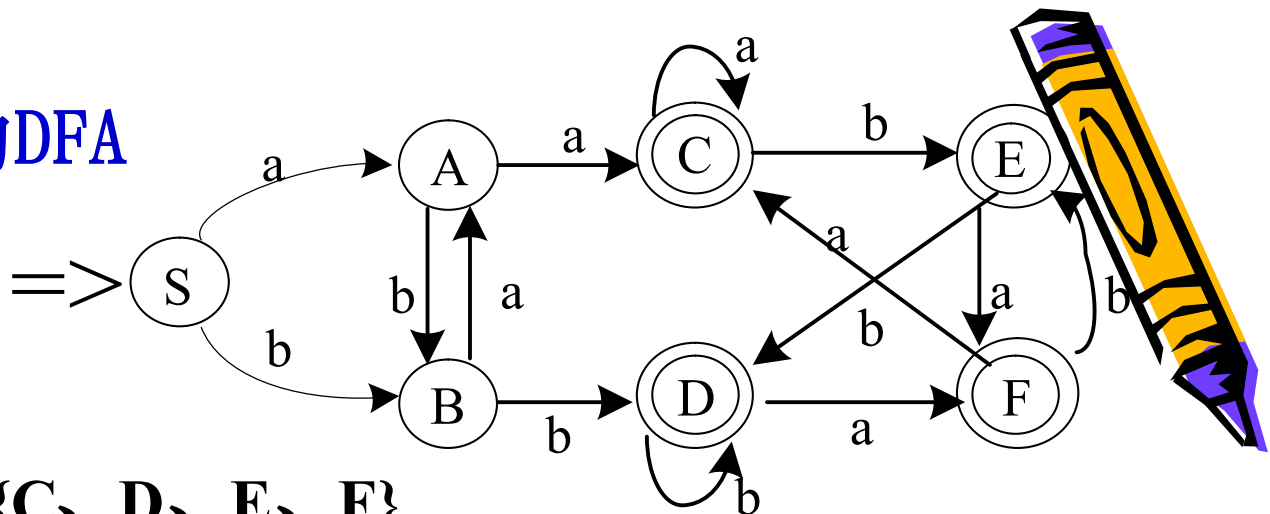


• 算法：

- 所有状态分成两个子集——**终态集**和**非终态集**；
- 运用判定状态**等价的原则**分别对两个子集的状态进行分析和划分，若发现某个状态与其它状态不等价，则将其作为一个新的状态子集，如果无法区分，则放在同一子集中；
- 从每个子集中选出一个状态做代表，即可构成简化的DFA；
- 含有原来初态的子集仍为**初态**，各终态的子集仍为**终态**。



例：化简下图的DFA



Π_0 : {S, A, B} {C, D, E, F}



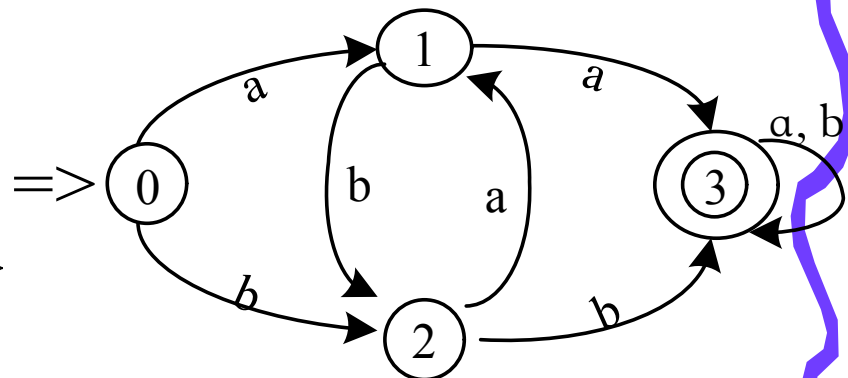
Π_1 : {A} {S, B} {C, D, E, F}



Π_2 : {A} {S} {B} {C, D, E, F}

选C为
代表
 $\downarrow$

{A} {S} {B} {C}



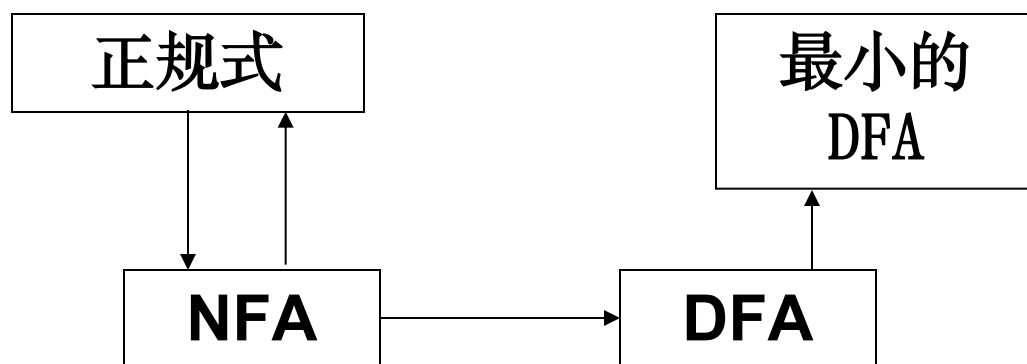
• 合并状态注意：

a、由于一个子集中，各状态等价，故只需将原进入该子集中各状态的弧都改为进入所选的状态，子集中各状态射出的弧均改为从该状态射出。

b、含有原来初态的子集仍为初态，含原终态的子集仍为终态。

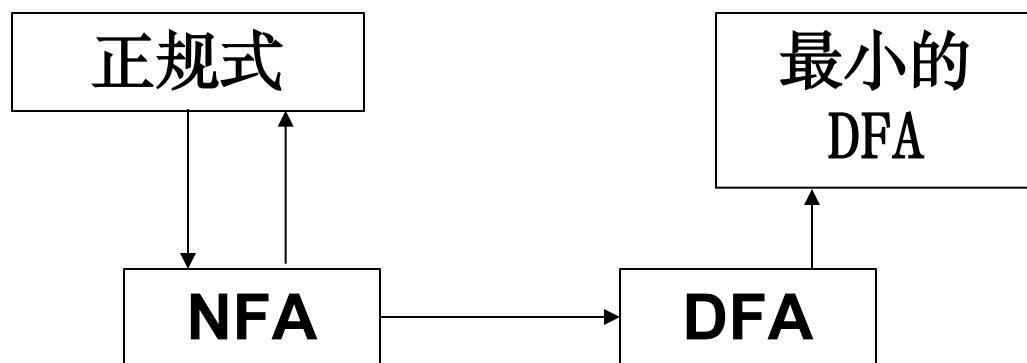


• 正规集的各种描述工具及其相互间的转换



小结

- 1 NFA确定化为DFA（子集法）
- 2 DFA化简（分割法）
- 3 正规集的各种描述工具及其相互间的转换



作业

教材79-81

6 (1) (2) (3)

7 (1) (2) (3)

8

